Randomized Consensus

Mohsen Lesani

FLP Theorem

There is no deterministic consensus protocol for asynchronous distributed systems when even one process crashes.

Common Coin

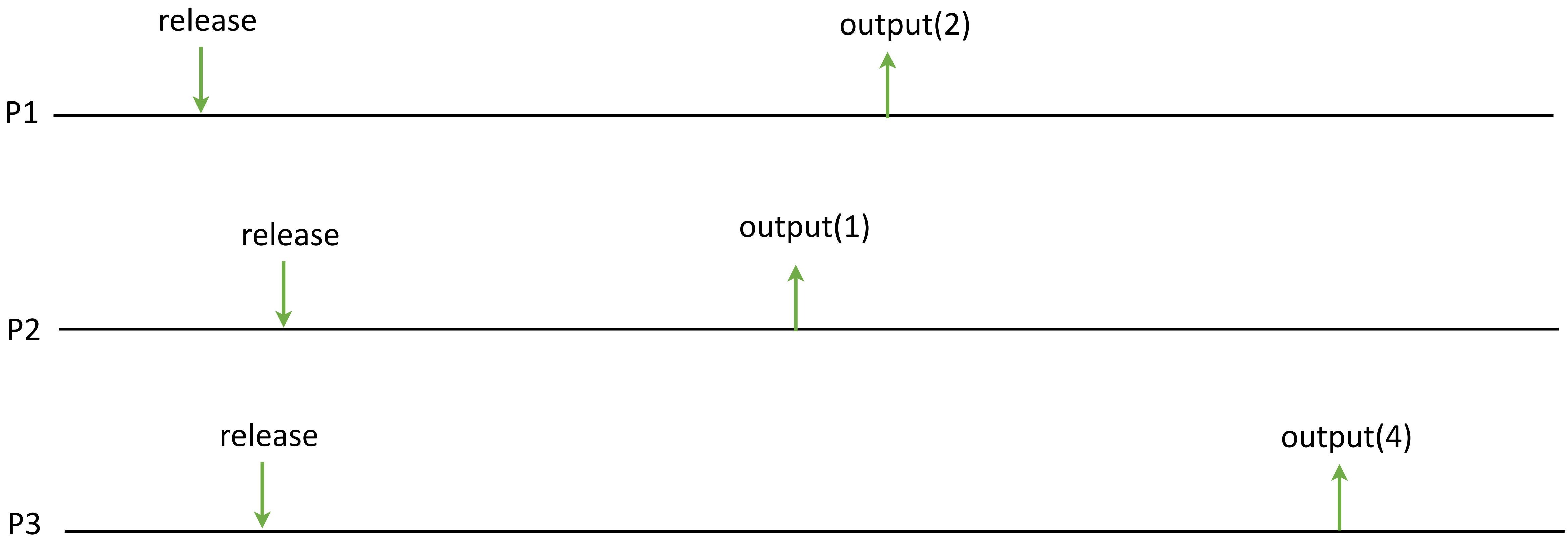
Random values from a domain D

- Request: $\langle$ coin, release $\rangle$:

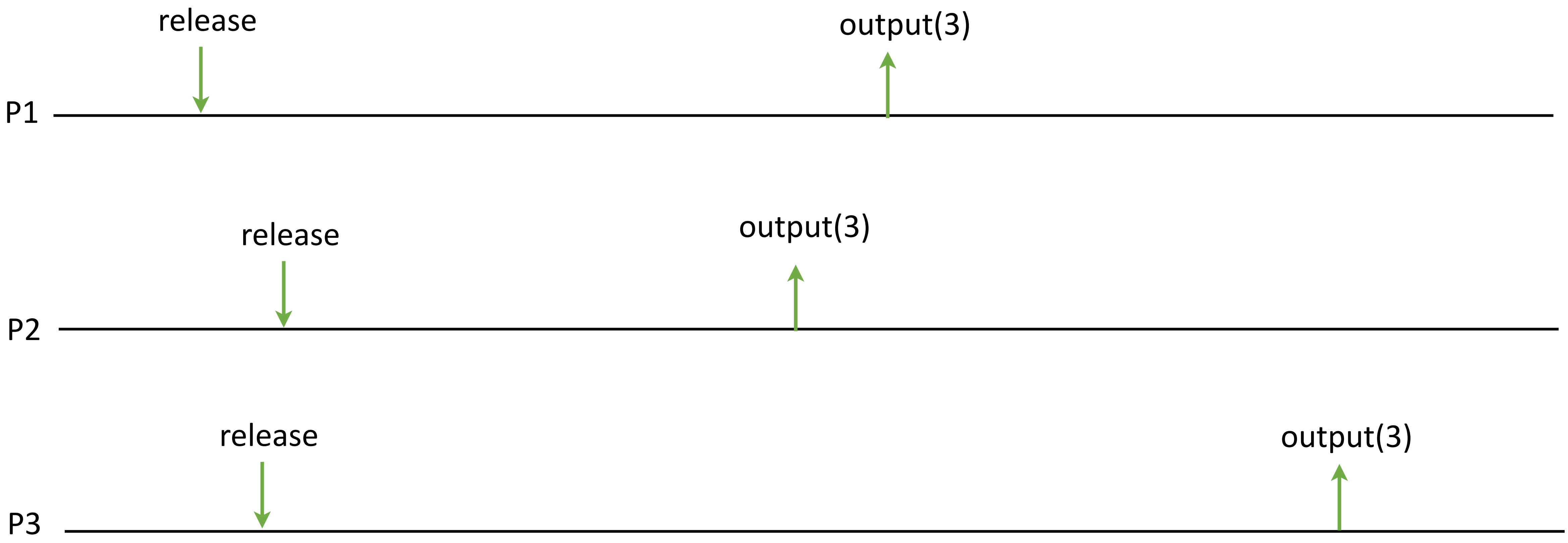
  Releases the coin.
- Indication: $\langle$ coin, output(b) $\rangle$:

  Outputs the coin value $v \in D$.

Common Coin Example



Common Coin Example, Eventually



We say that the coin eventually matches.

Common Coin Specification

- Matching:
  With probability at least δ, every correct process outputs the same coin value.
- No bias:
  In the event that all correct processes output the same coin value, the distribution of the coin is uniform over D (i.e., a matching coin outputs any value in D with probability $1/|D|$).
- Termination:
  Every correct process eventually outputs a coin value.

- Other properties …

Common Coin Implementation

Independent Choice Coin:
Upon releasing the coin, every process simply outputs a value locally at random from D according to the uniform distribution.

If the domain is one bit then this realizes a δ = 2^{-N+1} matching common coin, because the probability that every process selects some b is 2^{-N}, for two bits $b = \{0, 1\}$.

Common Coin Implementation

Beacon Coin:
An external trusted process, called the beacon, periodically chooses an unpredictable random value and broadcasts it at predefined times.
When an algorithm accesses a sequence of common coin abstractions, then for the k-th coin, every process outputs the k-th random value received from the beacon.

This coin always matches but it’s difficult to use it in an asynchronous system.

Randomized Consensus

- Agreement
- Integrity
- Validity
- **Probabilistic Termination:**
  Every correct process eventually decides some value with **probability 1**.

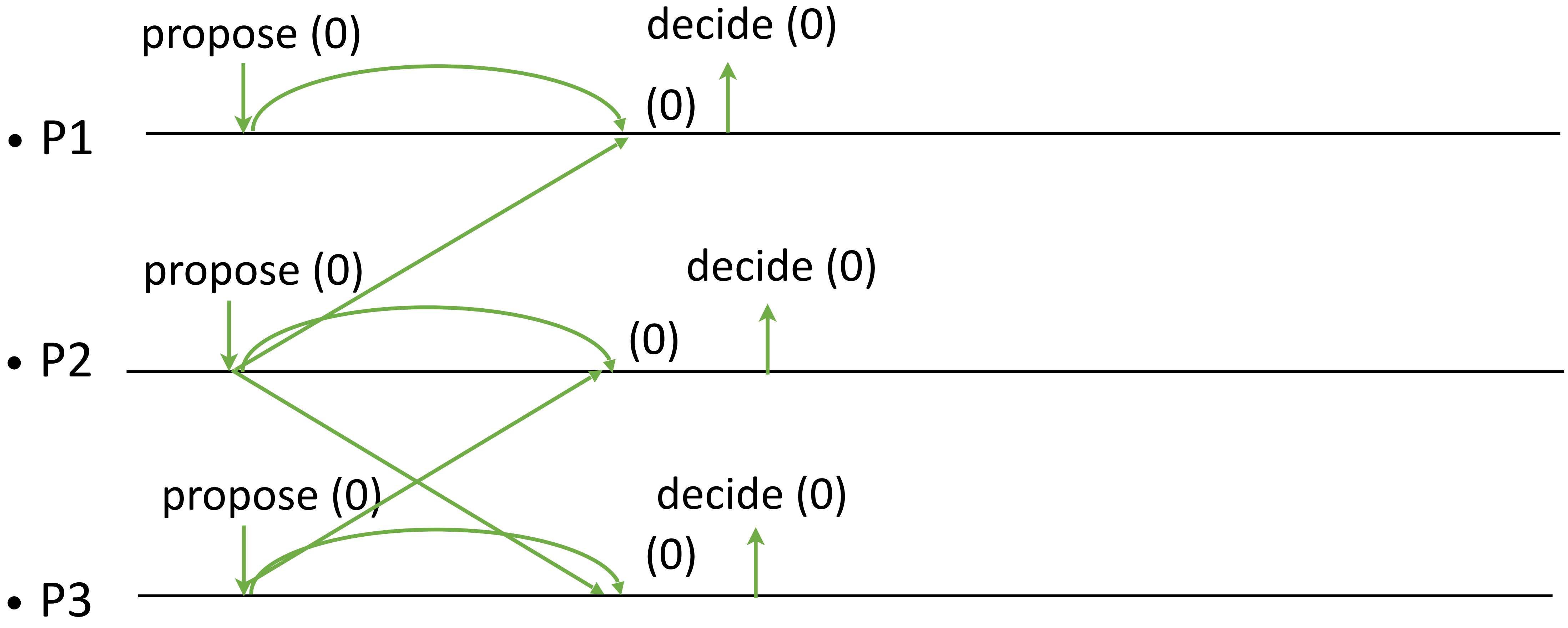
Observation

The common coin will eventually output the same value for all processes.

We need to design a consensus protocol that decides when all processes propose the same value.
Let’s consider binary consensus.
If the processes initially propose the same value, the protocol decides that value.
Otherwise, any value can be decided. We let the coin choose it.

Broadcast and wait for a quorum

Eventually all the proposals are the same.



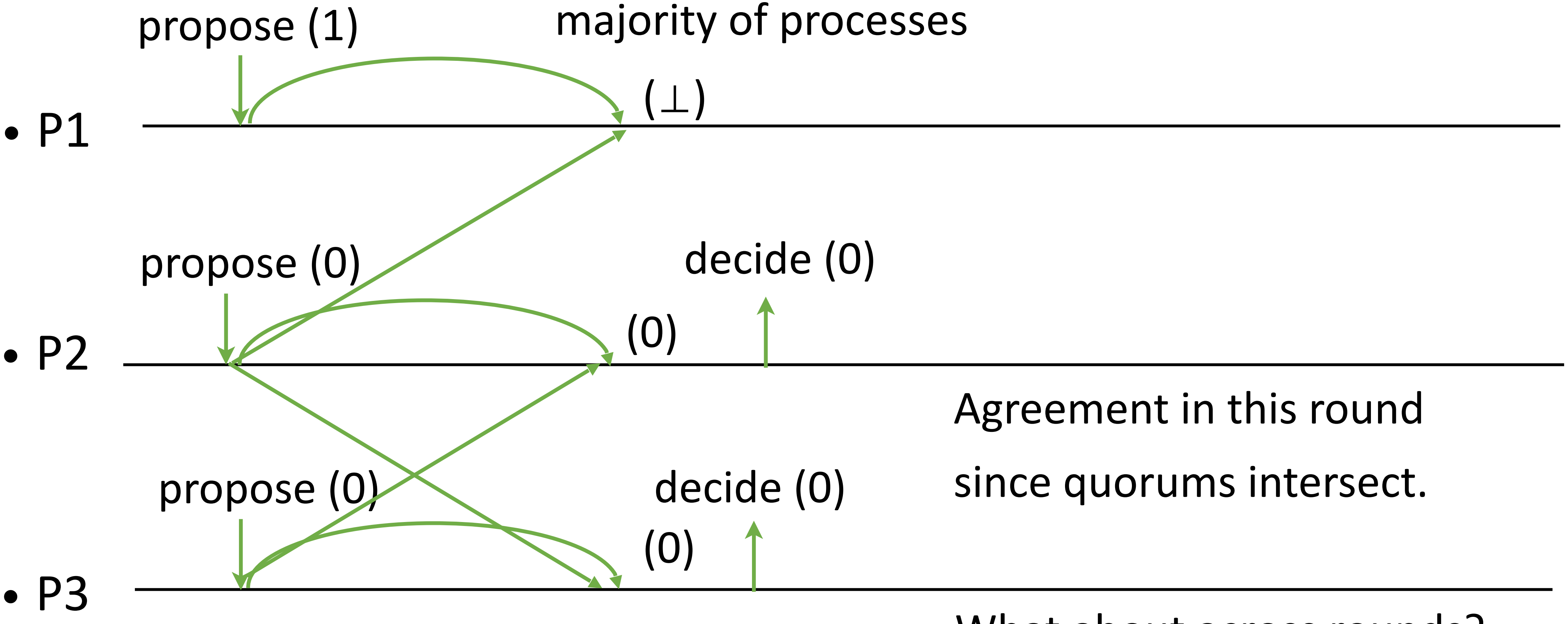
Does it have safety and liveness when proposals are different?

Broadcast and wait for a quorum

The proposals are different.

A majority (quorum) of processes are correct.

Can wait for at most a majority of processes

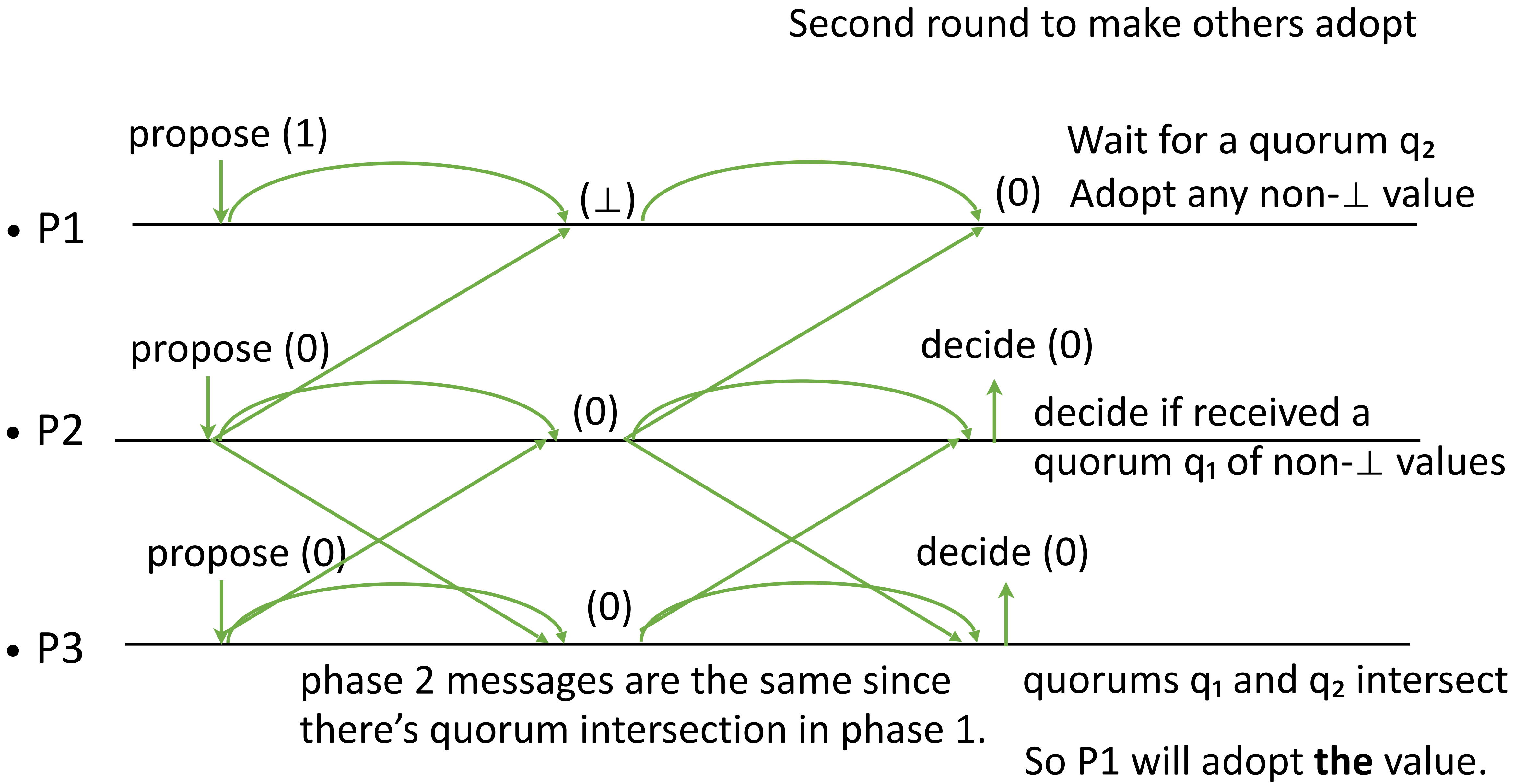


Agreement in this round

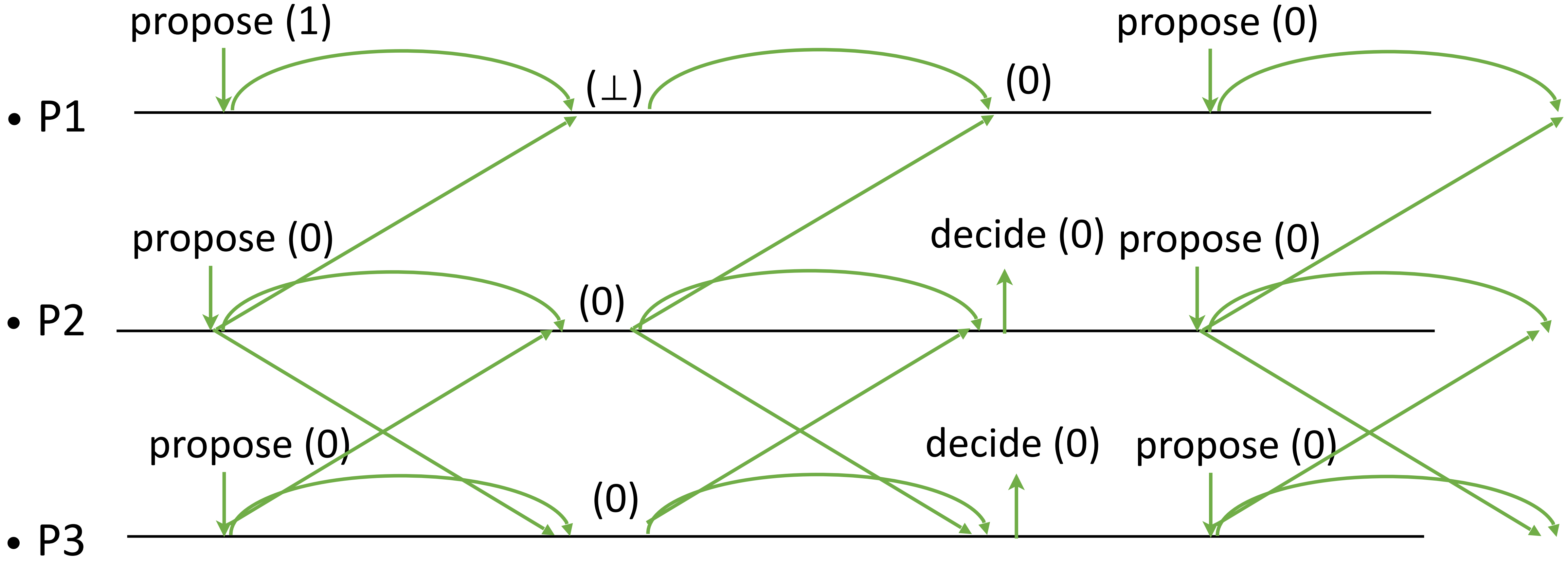
since quorums intersect.

What about across rounds?

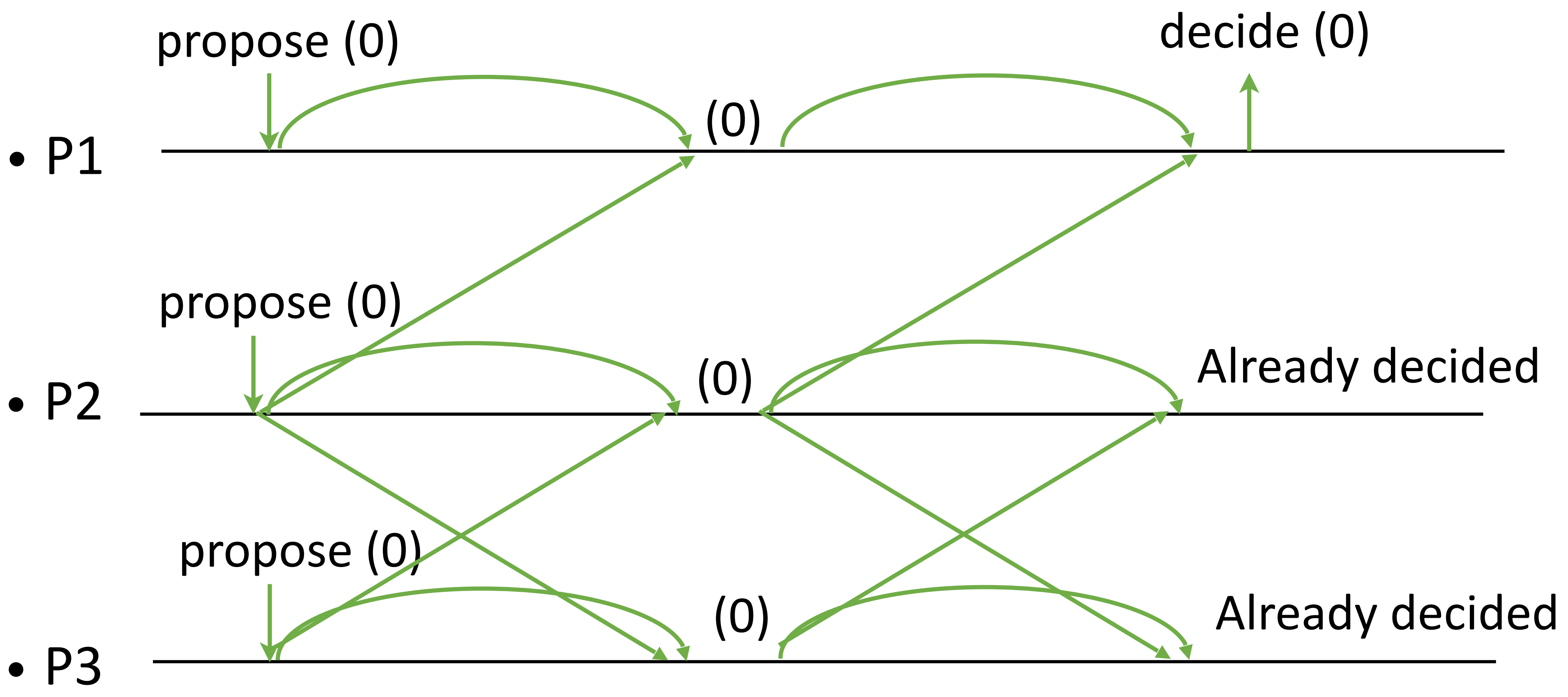
Adopt the decided value for next round



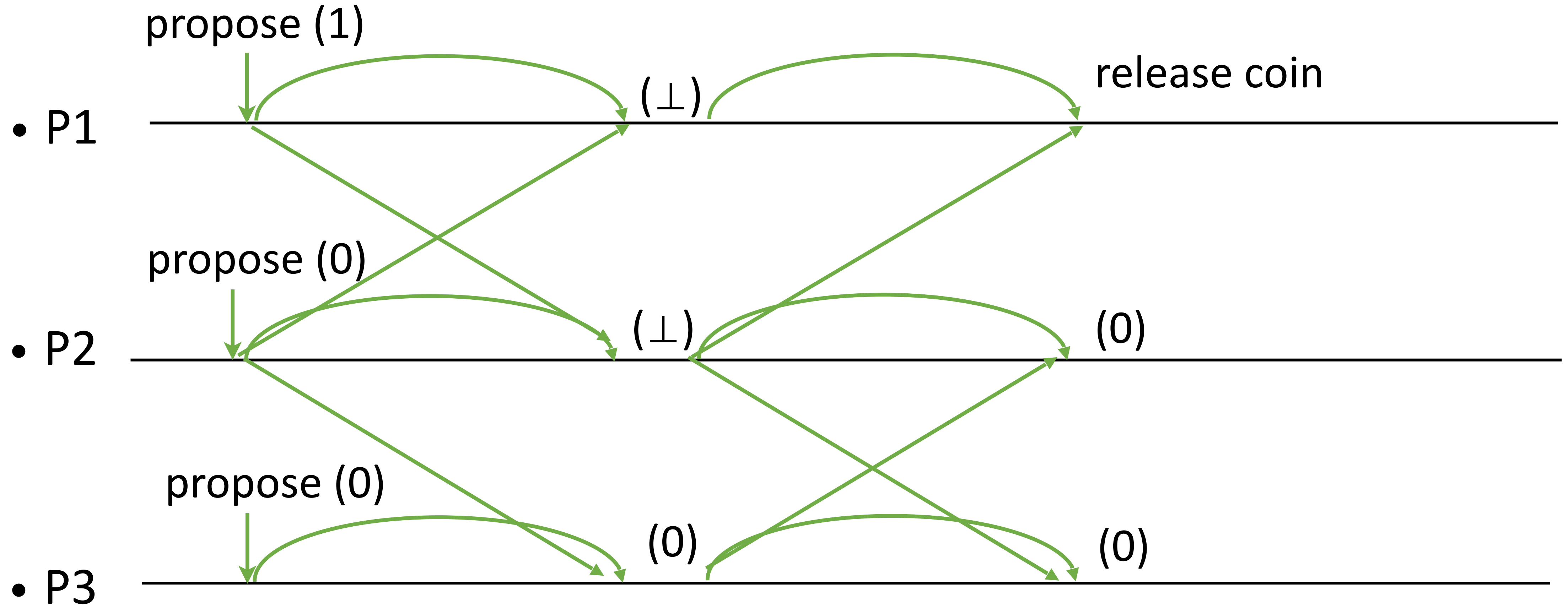
Adopt the decided value for next round



Adopt the decided value for next round

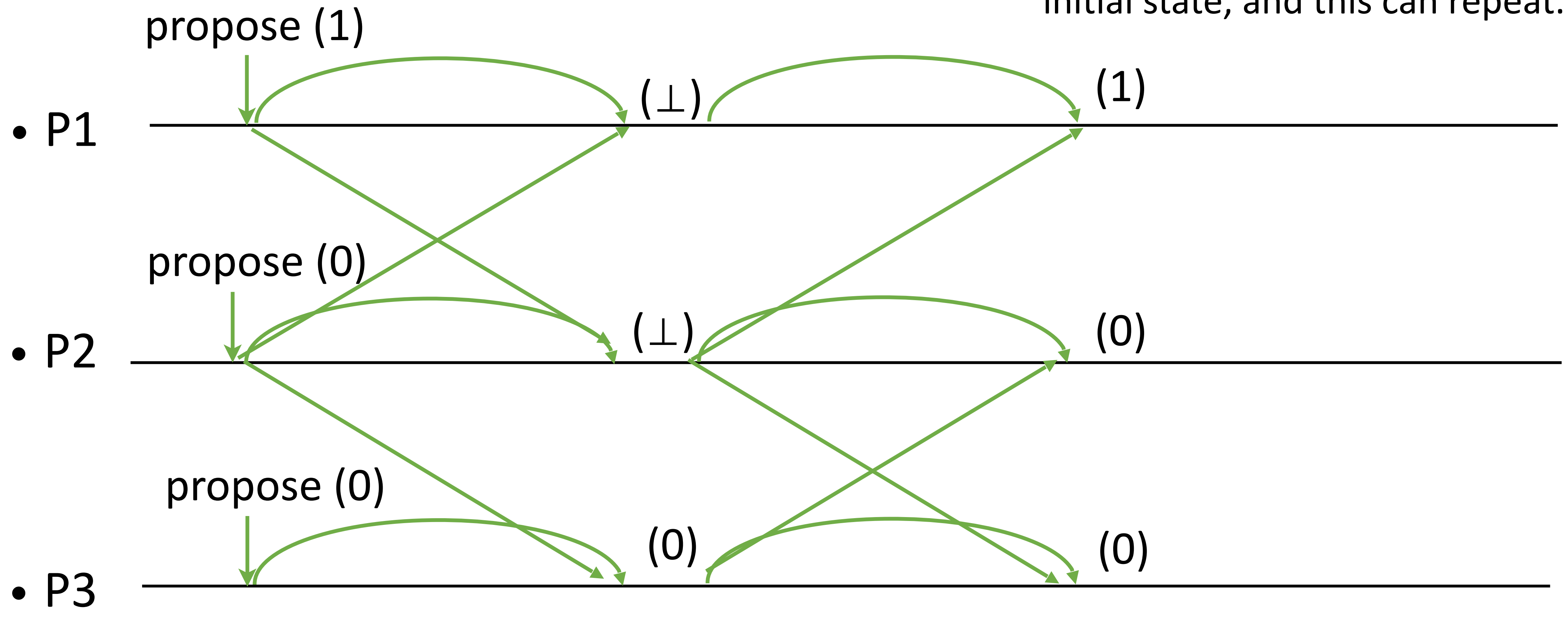


If not received non-⊥ value, use coin to get a value for next round



Deterministic choice would lead to non-termination

Deterministic choice of 1 for P1 takes the configuration to the initial state, and this can repeat.



Randomized Binary Consensus (1/2)

Implements: RandomizedConsensus, with domain {0, 1}.

Uses: BestEffortBroadcast, instance beb; CommonCoin (multiple instances).

upon event 〈 Init 〉 **do**

  round := 0; phase := 0; proposal := ⊥; decided := ⊥; val := $[\perp]^n$

upon event 〈 propose(v) 〉 **do**

  proposal := v; round := 1; phase := 1

  trigger 〈 beb, broadcast (Phase1(round, proposal)) 〉

upon event 〈 beb, deliver(p, Phase1(r, v) 〉 such that phase = 1 ∧ r = round **do**

  val[p] := v

  if decided ≠⊥ **then**

    trigger 〈 beb, broadcast(Phase2(round, decided)) 〉

  else if #(val) > n/2 **do**

    if ∃v≠⊥ such that #{p∈Π | val[p]=v} > n/2 **then**

      proposal := v

    else

      proposal := ⊥

    val := $[\perp]^n$; phase := 2

    trigger 〈 beb, broadcast(Phase2(round, proposal)) 〉

Randomized Binary Consensus (2/2)

upon event ⟨ beb, deliver(p, Phase2(r, v) ⟩ such that phase = 2 $\wedge$ r = round **do**

  val[p] := v

  if #(val) $\geq$ N − f $\wedge$ coin[round] is not initialized **then**

    Initialize a new instance coin[round] of CommonCoin with domain {0, 1}

    trigger ⟨ coin[round], release ⟩

upon event ⟨ coin[round], output(c) ⟩ **do**

  if decided $\neq \perp$ **then**

    proposal := decided

  else if $\exists v \neq \perp$ such that $\#\{p \in \Pi \mid val[p]=v\} > f$ **then**

    decided := v

    trigger ⟨ decide(v) ⟩

    proposal := v

  else if $\exists p \in \Pi, v \neq \perp$ such that val[p] = v **then**

    proposal := v

  else

    proposal := c

  val := $[\perp]^n$; round := round + 1; phase := 1

  trigger ⟨ beb, broadcast(Phase1(round, proposal)) ⟩

Add code to stop one round after decision.
Optimization: After decision, send the value to others by a reliable broadcast and stop.

Proof of Properties

- Agreement:
  Case decisions in the same round: because of quorum intersection in phase 1.
  Case decisions in two rounds: The value decided in the first round will be adopted by all processes for the second round.
- Integrity:
  The handler issues decision only if no decision is already made.
- Validity:
  A decided value in a round was proposed by a majority in that round.
  The proposal for the next round is either adopted in this round which is proposed by a majority in this round, or is from the coin. A value is taken from the coin only when the process receives $\perp$ from a majority in phase 2. So a process sent $\perp$ for phase 2 after receiving different binary values in phase 1. So any binary value from the coin is valid.
- Probabilistic Termination:
  The proposals adopted from the previous round are the same because of quorum intersection in phase 1. The coin takes values uniformly at random. It will eventually match, and all processes decide in that round.

Randomized Consensus for General Domains

- Common coin with a domain
- In order to satisfy validity, the coin should output only proposed values. The domain of the coin should be a subset of proposed values.
- In each round, each process broadcasts its proposal.
- Processes collect the set of proposals, and instantiate the coin with it.
- This set grows and processes eventually converge.
- Correct processes eventually invoke the coin with the same set.
- In the meanwhile the coin provides termination so that the protocol makes progress.

Randomized Consensus for General Domains

Implements: RandomizedConsensus.

Uses: …; ReliableBroadcast, instance rb

upon event 〈 Init 〉 **do**

...

values := ∅

upon event 〈 propose(v) 〉 **do**

...

values := values ∪ {v}

trigger 〈 rb, broadcast(Proposal(v) 〉

upon event 〈 rb, deliver(p, Proposal(v) 〉 **do**

values := values ∪ {v};

upon #(val) > N/2 ∧ phase = 2 ∧ decision = ⊥ **do**

Initialize a new instance coin[round] of CommonCoin with domain values;

…